PARADIGMAS DE LA PROGRAMACIÓN

Tarea 1

Sistema de gestión de rutinas de entrenamiento personalizadas

Autor:

Christopher Torrejón [a, b]

[a] Ingeniería Civil Industrial. Facultad de Ingeniería, Universidad Andrés Bello. Chile

[b] Ingeniería Civil Informática. Facultad de Ingeniería, Universidad Andrés Bello. Chile

c.torrejnallende@uandresbello.edu

21.774.845-3

Paradigmas de la Programación **[8025]**

Profesor encargado:

Juan Felipe Calderón Maureira

Abril 13, 2026. Valparaíso, Viña del Mar

Tabla de contenido

1. Introducción ... 1
1.1. Motivación ... 1
1.2. Objetivo ... 1
1.2.1. Objetivo General ... 1
1.2.2. Objetivos Específicos ... 1
1.3. Estructura del Documento ... 2
2. Descripción de la Solución ... 3
2.1. Modelo Propuesto ... 3
2.2. Clases Diseñadas ... 3
2.2.1. Clase Ejercicio ... 3
2.2.2. Clase Colección ... 4
2.2.3. Clase Rutina ... 6
2.2.4. Clase Colección_rutina ... 7
2.2.5. Clase Orquestador ... 7
2.3. Relaciones entre clases ... 9
2.3.1. Figura 1. Diagrama de clases ... 9
2.4. Justificación ... 9
3. Conclusión ... 11
3.1. Reflexión ... 11
3.2. Análisis de resultados y objetivos ... 11
3.2.1. Resolución del Objetivo General ... 11
3.2.2. Objetivo específico 1 ... 12
3.2.3. Objetivo específico 2 ... 12
3.2.4. Objetivo específico 3 ... 12
4. Referencias Bibliográficas ... 13
4.1. Referencias textuales ... 13

1. Introducción

1.1. Motivación

Una proporción significativa de estudiantes universitarios comparte una motivación fundamental durante su proceso formativo: la adquisición de conocimientos que puedan ser aplicados de manera concreta en la resolución de problemas reales. En el caso de los estudiantes de ingeniería civil informática, este propósito adquiere especial relevancia en el ámbito de la programación, disciplina que se sustenta en diversos paradigmas orientados a la abstracción y modelamiento de situaciones del mundo real mediante código.

El dominio de estos paradigmas, así como la capacidad de identificar el más adecuado según el contexto, constituye una herramienta clave para el futuro desempeño profesional. En este sentido, la presente actividad se marca en la aplicación práctica del paradigma de la programación orientada a objetos, el cual permite estructurar soluciones eficientes a problemas complejos.

Particularmente, este informe aborda la problemática asociada a un centro de entrenamiento deportivo ficticio, en el cual los instructores deben diseñar rutinas personalizadas para sus clientes en función de sus objetivos físicos. Dado que cada cliente presenta necesidades específicas, las rutinas deben ser adaptadas individualmente, considerando distintos tipos de ejercicios y niveles de intensidad.

Para dar solución a este problema, se implementa un modelo basado en programación orientada a objetos utilizando el lenguaje C++, permitiendo abstraer adecuadamente los requerimientos y restricciones del sistema propuesto.

1.2. Objetivo

1.2.1. Objetivo General

- Diseñar e implementar un modelo basado en programación orientada a objetos en C++, representado mediante un diagrama de clases, que permita gestionar y generar rutinas de entrenamiento personalizadas para clientes de un centro deportivo.

1.2.2. Objetivos Específicos

1. Implementar las funcionalidades principales del sistema, incluyendo la creación, actualización, eliminación, consulta y búsqueda de ejercicios, así como la generación de rutinas de entrenamiento.
2. Aplicar el uso de punteros en C++ para la gestión dinámica de memoria y la manipulación eficiente de objetos dentro de estructuras como vectores.
3. Analizar el uso de herramientas de inteligencia artificial como apoyo en el proceso de desarrollo, particularmente en la identificación y resolución de problemas asociados a la gestión de memoria, como el manejo de punteros colgantes.

1.3. Estructura del Documento

El presente informe se organiza en distintas secciones con el propósito de exponer de manera clara y ordenada el desarrollo de la solución propuesta.

En primer lugar, se presenta la **introducción**, donde se aborda la motivación del trabajo, el objetivo general y los objetivos específicos que guiarán el desarrollo del proyecto.

Posteriormente, se desarrolla la **descripción de la solución**, en la cual se exponen la problemática planteada, el modelo propuesto y las clases diseñadas, junto con sus respectivas relaciones. Asimismo, se incluye el diagrama de clases que representa gráficamente la arquitectura del sistema, además de la justificación de las principales decisiones de diseño adoptadas.

Finalmente, se presenta la **conclusión**, donde se realiza una reflexión sobre el trabajo desarrollado, se evalúa el cumplimiento de los objetivos propuestos y se destacan los principales aprendizajes obtenidos durante la elaboración del proyecto.

2. Descripción de la Solución

2.1. Modelo Propuesto

El problema abordado en el presente trabajo se sitúa en el contexto de un centro de entrenamiento deportivo, donde los instructores deben diseñar rutinas de entrenamiento personalizadas para sus clientes en función de sus objetivos físicos y nivel de condición. Cada rutina está compuesta por un conjunto variable de ejercicios, los cuales pueden clasificarse principalmente en dos categorías: ejercicios cardiovasculares y ejercicios de fuerza.

Cada ejercicio posee atributos relevantes para su correcta gestión, tales como un identificador único, nombre, tipo, nivel de intensidad, tiempo estimado de ejecución, descripción y la última semana en la que fue utilizado. A su vez, los niveles de intensidad se encuentran predefinidos en cuatro categorías: básico, intermedio, avanzado y alto rendimiento, lo que condiciona tanto la selección de ejercicios como la exigencia de las rutinas generadas.

Una de las principales restricciones del sistema consiste en evitar la repetición de ejercicios en rutinas asignadas a un mismo cliente en semanas consecutivas, lo que implica la necesidad de llevar un registro histórico de uso. Además, el sistema debe ser capaz de gestionar de manera eficiente un conjunto dinámico de ejercicios, permitiendo operaciones como la creación, actualización, eliminación, consulta y búsqueda según criterios específicos.

En este contexto, surge la necesidad de desarrollar una solución que permita modelar adecuadamente los distintos elementos involucrados, así como sus relaciones y restricciones, con el objetivo de facilitar la generación de rutinas coherentes, personalizadas y consistentes con los requerimientos del sistema.

2.2. Clases Diseñadas

En el contexto de la solución propuesta, se definió una estructura modular basada en el uso de Múltiples archivos, destacando un archivo de cabecera (.h), el cual contiene la declaración de todas las clases utilizadas en el sistema, incluyendo sus atributos y métodos asociados. Esta separación permite una mejor organización del código y facilita su mantenimiento y escalabilidad.

2.2.1. Clase Ejercicio

En primer lugar, se identificó que un ejercicio, si bien es un concepto intangible en el mundo real, posee un conjunto de características que pueden ser abstraídas y representadas mediante una estructura de datos. A partir de esta interpretación, se define la clase **Ejercicio**, la cual constituye la base del modelo propuesto.

Dicha clase contempla los siguientes atributos:

- int codigo_identificador
- string nombre
- string tipo_ejercicio
- string nivel_intensidad
- int tiempo_estimado
- string descripcion
- int ultima_semana

Estos atributos fueron encapsulados utilizando el modificador de acceso **privado**, con el objetivo de resguardar la integridad de los datos almacenados en cada instancia de la clase. En consecuencia, se implementaron métodos *getter* y *setter* para cada uno de los atributos, permitiendo su acceso y modificación de manera controlada.

Además, la clase **Ejercicio** cuenta con un constructor, encargado de inicializar los atributos al momento de la creación del objeto, y un destructor, orientado a la correcta liberación de recursos.

Por otra parte, se definieron tres métodos virtuales, lo que permite la aplicación del principio de polimorfismo en clases derivadas:

- virtual void mostrar(): encargado de desplegar en consola los valores de los atributos del objeto, utilizando un formato estructurado.
- virtual void filtrar(string tipo_ejercicio): permite asignar valores a los atributos del objeto a partir de la entrada del usuario, asegurando una correcta manipulación de tipos de datos.
- virtual void modificar(): habilitar la modificación de ciertos atributos del objeto, excluyendo aquellos que no deben ser alterados, tales como el código identificador, el tipo de ejercicio y la última semana de uso.

La clase **Ejercicio** actúa como una clase base (o clase padre) para dos clases derivadas: **Ejercicio_Cardiovascular** y **Ejercicio_Fuerza**. Ambas heredan los atributos y métodos de la clase principal, extendiendo su funcionalidad mediante atributos específicos:

- **Ejercicio_Cardiovascular** incorpora el atributo:
  - string tipo_maquina
- **Ejercicio_Fuerza** incorpora el atributo:
  - string grupo_muscular

Cada una de estas clases incluye sus respectivos métodos *getter* y *setter*, permitiendo la gestión de sus atributos particulares.

La creación de estas clases derivadas responde a la necesidad de representar de manera más precisa los distintos tipos de ejercicios definidos en la problemática. De este modo, se logra una mejor organización del modelo, permitiendo diferenciar comportamientos y características específicas según el tipo de ejercicio, lo que facilita la extensibilidad del sistema ante futuros requerimientos.

2.2.2. Clase Colección

Una vez definida la estructura que permite representar un ejercicio y sus respectivas especializaciones, surge una problemática fundamental en el diseño del sistema: la necesidad de almacenar y gestionar múltiples instancias de estos objetos de manera eficiente. En este contexto, se hace necesario incorporar una estructura que permita organizar dinámicamente la información generada por el usuario.

Para abordar este requisito, recurra al uso de **vectores**. Desde un punto de vista conceptual, un vector puede entenderse como una entidad que posee magnitud, dirección y sentido, siendo ampliamente utilizado en la representación de fenómenos físicos (J. Fernández, s.f.). Esta noción puede reinterpretarse en el ámbito de la programación como una estructura que permite acceder y organizar elementos en memoria de forma secuencial, manteniendo una “dirección” hacia los datos almacenados.

En el lenguaje C++, un vector corresponde a una estructura dinámica que permite almacenar múltiples elementos de manera contigua en memoria (Geeks for Geeks, 2025). En el contexto del presente sistema, esta característica resulta especialmente útil para gestionar colecciones de objetos, ya que permite almacenar referencias a instancias de la clase **Ejercicio**.

A partir de esta necesidad, se define la clase **Colección**, la cual tiene como propósito principal almacenar y gestionar un conjunto de ejercicios. Esta clase presenta el siguiente atributo encapsulado:

- vector<Ejercicio*> ejercicios

El uso de punteros dentro del vector permite trabajar con objetos dinámicos en memoria, facilitando la manipulación eficiente de los mismos sin necesidad de realizar copias innecesarias.

En cuanto a su interfaz pública, la clase **Colección** dispone de métodos *getter* y *setter* que permiten acceder y modificar el contenido del vector, respectivamente. Estos métodos aseguran una interacción controlada con la estructura interna de la clase.

Además, se implementaron diversos métodos dentro de la clase **Colección**, orientados a la gestión dinámica de los ejercicios almacenados.

En primer lugar, se encuentra el método:

- void eliminar_elemento(int indice)

Este método tiene como finalidad eliminar un ejercicio específico del vector a partir de su posición. Para ello, no solo se remueve el elemento de la estructura, sino que también se libera correctamente la memoria asociada al objeto mediante el uso de *delete*, evitando así problemas relacionados con fugas de memoria y asegurando una gestión eficiente de los recursos.

Por otra parte, se incorporó el método:

- void mostrar_ejercicio_tipo_nivel(string nivel_intensidad)

Este método permite visualizar únicamente aquellos ejercicios cuyo nivel de intensidad coincida con el valor recibido como parámetro. De esta manera, se facilita la segmentación de los datos, permitiendo al usuario acceder de forma directa a ejercicios clasificados, por ejemplo, como “básico”, “intermedio” o “avanzado”, mejorando la experiencia de uso y la organización de la información.

Finalmente, se implementó el método:

- int filtrador()

Este método tiene como propósito mostrar en pantalla todos los ejercicios disponibles dentro de la colección, permitiendo al usuario seleccionar uno de ellos para trabajar. Una vez realizada la elección, el método retorna el índice del ejercicio seleccionado, el cual es posteriormente utilizado por otras funciones del sistema para operar sobre dicho elemento de manera específica.

Cabe destacar que la clase Colección no cuenta con un constructor parametrizado, lo cual responde a su naturaleza como contenedor dinámico. Su propósito es almacenar objetos de manera progresiva durante la ejecución del programa, por lo que resulta más adecuada instanciarla de forma genérica y poblarla mediante posteriormente propósito la incorporación de elementos a través de métodos específicos, permitiendo así una gestión flexible de los ejercicios.

2.2.3. Clase Rutina

Una vez resulta la gestión de ejercicios individuales y su almacenamiento, se aborda la necesidad de representar el elemento central del problema: la rutina de entrenamiento. Esta entidad corresponde a una agrupación de ejercicios asociados a un cliente específico, y cuyo propósito es cumplir con determinados objetivos físicos en un período de tiempo definido.

A partir de esta necesidad, se define la clase **Rutina**, la cual modela una rutina de entrenamiento como una colección estructurada de ejercicios junto con información adicional relevante. Esta clase presenta los siguientes atributos:

- vector<Ejercicio*> ejercicios
- string cliente
- int tiempo_total

El atributo **ejercicios** corresponde a un vector de punteros a objetos de tipo **Ejercicio**, el cual almacena exclusivamente aquellos ejercicios que forman parte de una rutina en particular. A diferencia de la clase **Colección**, donde el vector actúa como un repositorio general de ejercicios, en este caso cumple una función específica, asociando un subconjunto de ejercicios a una instancia concreta de rutina.

Por su parte, el atributo **cliente** permite identificar al usuario al cual se le asigna la rutina. Se opta por definirlo como una cadena de caracteres, considerando que los identificadores de clientes pueden contener combinaciones alfanuméricas.

El atributo **tiempo_total** representa la duración total estimada de la rutina, la cual se calcula como la suma de los tiempos individuales de los ejercicios que la componen. Este valor se actualiza dinámicamente a medida que se agregan ejercicios a la rutina.

Al igual que en las clases previamente definidas, los atributos se encuentran encapsulados mediante el modificador de acceso **privado**, y se dispone de métodos *getter* y *setter* para su manipulación controlada.

Además, la clase **Rutina** implementa un destructor, cuyo objetivo es liberar correctamente la memoria asociada a los objetos almacenados en el vector de ejercicios. Esto resulta especialmente relevante dado el uso de memoria dinámica mediante punteros, evitando así posibles fugas de memoria.

Cabe destacar que no se definió un constructor parametrizado para esta clase. Esta decisión de diseño responde a que las rutinas se construyen de manera progresiva durante la ejecución del programa, en función de la información proporcionada por el usuario (cliente, cantidad de ejercicios y nivel de intensidad). Por lo tanto, resulta más adecuado inicializar una instancia de rutina de forma genérica y posteriormente asignar sus atributos mediante el uso de métodos *setter*.

En conjunto, la clase **Rutina** permite modelar de manera eficiente la relación entre clientes y ejercicios, constituyendo un componente fundamental para la generación de rutinas personalizadas dentro del sistema.

2.2.4. Clase Colección_rutina

Finalmente, se define la clase **Colección_rutina**, cuya finalidad es gestionar el almacenamiento de las rutinas generadas durante la ejecución del programa. Esta clase cumple un rol análogo al de la clase **Colección**, pero orientado específicamente a objetos de tipo **Rutina**.

Para ello, se define el siguiente atributo encapsulado:

- vector<Rutina*> rutinas

Este vector permite almacenar dinámicamente las distintas rutinas creadas por el usuario, facilitando su organización y posterior acceso. Al igual que en el caso de la clase **Colección**, el uso de punteros permite trabajar con objetos dinámicos en memoria, optimizando la gestión de recursos y evitando copias innecesarias.

En cuanto a su interfaz pública, la clase dispone de métodos *getter* y *setter* que permiten acceder y modificar el contenido del vector de manera controlada.

Además, se implementa un destructor, cuyo propósito es liberar correctamente la memoria asociada a cada una de las rutinas almacenadas en el vector. Esta consideración resulta fundamental debido al uso de memoria dinámica, asegurando así la correcta finalización del programa sin incurrir en fugas de memoria.

Cabe destacar que la clase **Coleccion_rutina** no cuenta con un constructor parametrizado, decisión que se fundamenta en su rol como contenedor dinámico. En este sentido, la colección se instancia de forma genérica y se va poblando progresivamente a medida que el usuario genera nuevas rutinas durante la ejecución del sistema.

En conjunto, esta clase permite complementar el modelo propuesto, proporcionando una estructura organizada para la gestión de rutinas, y consolidando así la arquitectura general del sistema.

2.2.5. Clase Orquestador

Con el objetivo de centralizar la lógica del sistema y mejorar la organización del código, se implementó la clase **Orquestador**, la cual actúa como el núcleo del programa. Esta clase es responsable de gestionar la interacción entre las distintas clases, coordinar la creación de objetos y asegurar el correcto flujo de ejecución.

En este sentido, el **Orquestador** se encarga de instanciar y administrar las colecciones principales del sistema, garantizando que los ejercicios y rutinas sean almacenados de manera consistente. Asimismo, concentra la lógica de control que anteriormente se encontraba dispersa, permitiendo una estructura más modular, mantenible y escalable.

Para cumplir con estas funciones, la clase cuenta con los siguientes métodos:

- void menu_principal()

Este método tiene un propósito principalmente visual, ya que se encarga de desplegar en pantalla el menú de opciones disponibles para el usuario. A través de este menú, se estructura la interacción con el sistema, permitiendo acceder a las distintas funcionalidades implementadas.

- string pedir_intensidad()

Este método permite al usuario seleccionar el nivel de intensidad con el cual desea trabajar. Para ello, se presenta un menú interactivo con distintas opciones, tales como “básico”, “intermedio”, “avanzado” y “alto rendimiento”. Una vez realizada la selección, el método retorna el valor correspondiente, el cual se utiliza posteriormente para filtrar ejercicios y construir rutinas acordes a dicho nivel.

- Coleccion* ejercicios_disponibles(Coleccion* ejercicios, Coleccion_rutina* rutinas, int semana, string cliente)

Este método cumple un papel fundamental en la personalización de rutinas. Su objetivo es devolver una porción de la colección original de ejercicios, excluyendo aquellas que ya fueron utilizadas previamente por un mismo cliente. De esta forma, si un usuario ha trabajado con un conjunto específico de ejercicios en una semana anterior, estos son descartados en la generación de una nueva rutina. Esto permite evitar la repetición innecesaria y fomentar una mayor variabilidad en los entrenamientos.

- void coordinador()

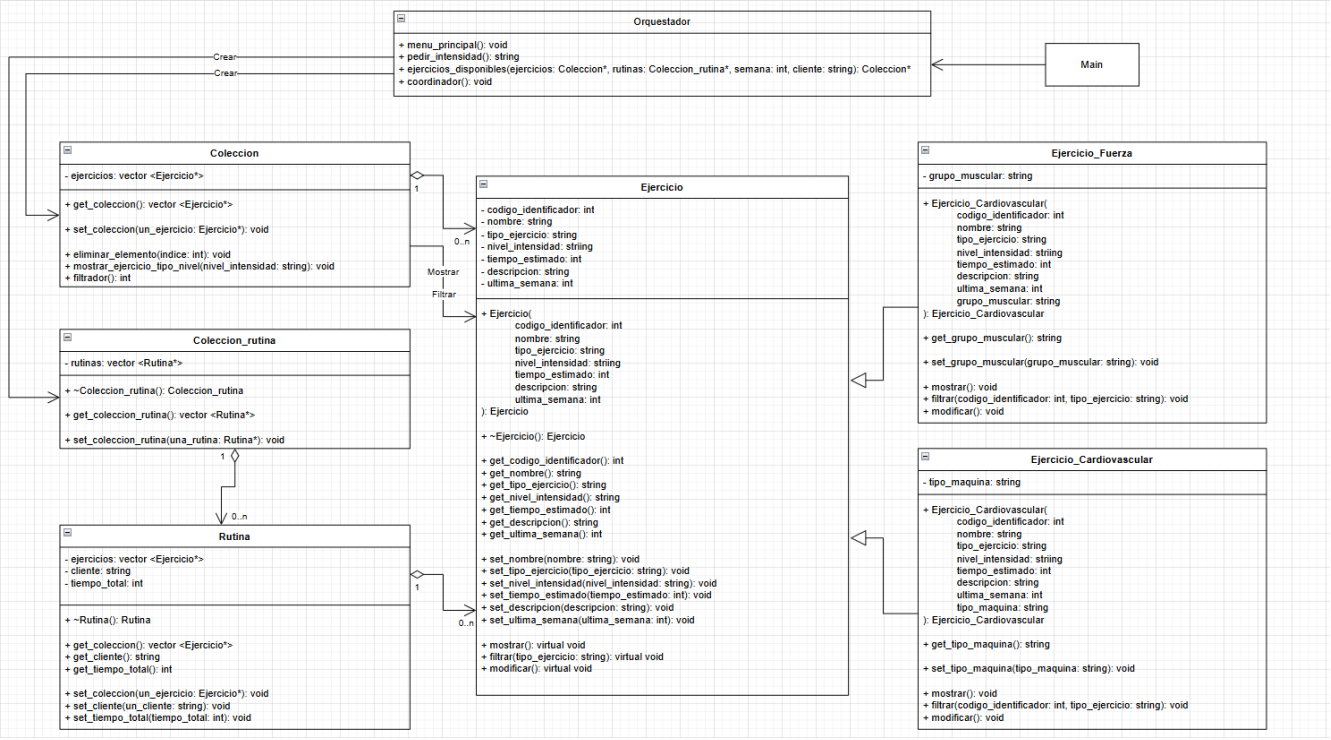
Este método corresponde al punto central de ejecución del programa, concentrando la lógica principal que anteriormente se encontraba en la función *main*. Su incorporación dentro de la clase **Orquestador** responde a la necesidad de encapsular el control del flujo del programa en una entidad única, reforzando así el principio de responsabilidad única y mejorando la claridad estructural del sistema.

En conjunto, la clase **Orquestador** permite desacoplar la lógica de control de las estructuras de datos, facilitando tanto la comprensión como la futura extensión del programa.

2.3. Relaciones entre clases

A partir del diagrama de clases propuesto, es posible identificar las distintas relaciones que estructuran el modelo del sistema.

2.3.1. Figura 1. Diagrama de clases



Nota. La **Figura 1**, fue creada con la herramienta *draw.io* para la correcta representación de la solución propuesta.

2.4. Justificación

El diagrama de clases propuesto representa la estructura general del sistema y la forma en que interactúan sus distintos componentes. A continuación, se describe su funcionamiento siguiendo el flujo de ejecución del programa.

En primer lugar, se encuentra la función *main*, cuyo propósito es ejecutar exclusivamente el programa. Esta no contiene lógica de negocio, ya que su única responsabilidad es instanciar un objeto de la clase **Orquestador**, denominado *programa*, y delegar en él el control total de la aplicación mediante la llamada al método *coordinador()*. Esta decisión permite desacoplar la ejecución inicial de la lógica interna del sistema, favoreciendo una mejor organización del código.

Una vez iniciado el flujo dentro del **Orquestador**, se crean las dos colecciones principales del sistema: **Coleccion** y **Coleccion_rutina**. Estas estructuras cumplen roles fundamentales en la gestión de la información.

La clase **Coleccion** actúa como una base de datos central encargada de almacenar todos los ejercicios creados durante la ejecución del programa. Esta colección puede contener desde cero hasta *n* ejercicios, lo que refleja su naturaleza dinámica y flexible. Además, es capaz de almacenar objetos de tipo **Ejercicio**, independientemente de si corresponden a ejercicios de fuerza o cardiovasculares.

Lo anterior es posible gracias al uso de **herencia**, donde **Ejercicio** actúa como clase base, mientras que **Ejercicio_Fuerza** y **Ejercicio_Cardiovascular** corresponden a clases derivadas que extienden su funcionalidad. Cada una de estas clases hijas incorpora atributos y comportamientos específicos, como el grupo muscular en el caso de fuerza o el tipo de máquina en los ejercicios cardiovasculares.

A su vez, se aplica el principio de **polimorfismo**, ya que la colección almacena punteros de tipo **Ejercicio**, permitiendo que objetos de distintas clases derivadas sean tratados de manera uniforme. Esto posibilita, por ejemplo, recorrer la colección y ejecutar métodos como *mostrar()* o *filtrar()* sin necesidad de distinguir explícitamente el tipo concreto del ejercicio, delegando dicho comportamiento a la implementación específica de cada clase hija.

Además, la clase **Colección** incorpora métodos que permiten operar sobre el conjunto de ejercicios almacenados. Entre ellos destacan *mostrar_ejercicio_tipo_nivel(...)*, que filtra y muestra ejercicios según su nivel de intensidad, y *filtrador()*, que presenta todos los ejercicios disponibles al usuario y retorna el índice del ejercicio seleccionado, permitiendo su posterior utilización dentro del sistema.

Por otra parte, la clase **Coleccion_rutina** tiene como objetivo gestionar las rutinas creadas por los usuarios. Al igual que la colección de ejercicios, esta puede contener desde cero hasta *n* rutinas, adaptándose dinámicamente a la cantidad de información generada durante la ejecución.

En este contexto, una **Rutina** corresponde a una agrupación de ejercicios definidos por el usuario, junto con atributos adicionales como el cliente asociado y el tiempo total estimado. Por ejemplo, una rutina puede estar compuesta por un subconjunto de ejercicios seleccionados desde la colección principal, permitiendo construir entrenamientos personalizados a partir de los ejercicios previamente registrados.

De esta forma, la interacción entre **Coleccion** y **Coleccion_rutina** permite separar claramente la gestión de ejercicios individuales de la construcción de rutinas, promoviendo una arquitectura modular y coherente. En conjunto, el **Orquestador** actúa como el nexo que coordina estas estructuras, asegurando la correcta creación, selección y organización de los datos dentro del sistema.

Finalmente, el diseño implementado se fundamenta en principios clave de la programación orientada a objetos, tales como encapsulamiento, herencia y polimorfismo, los cuales permiten construir un sistema flexible, reutilizable y fácilmente mantenible.

3. Conclusión

3.1. Reflexión

El desafío planteado proporciona una base sólida para el desarrollo del pensamiento crítico en la abstracción de problemas del mundo real mediante el uso de la programación orientada a objetos en C++. La necesidad de modelar una problemática concreta, junto con la implementación de una solución funcional, permite consolidar conceptos fundamentales de diseño y estructuración de software.

Asimismo, el hecho de trabajar en un lenguaje como C++, caracterizado por su cercanía al manejo de memoria y su menor nivel de abstracción en comparación con otros lenguajes, representa un valor agregado significativo. Este entorno, que puede resultar desafiante en etapas iniciales, contribuye a la comprensión profunda de conceptos como el uso de punteros, los cuales suelen constituir una de las principales dificultades para programadores en formación. En este sentido, el desarrollo del proyecto permite disminuir la complejidad

Por otra parte, si bien el tiempo asignado para la realización del trabajo resulta suficiente para completar los requerimientos propuestos, es importante destacar que la correcta implementación de la solución demanda una inversión considerable de horas de trabajo. Esto se debe tanto a la complejidad del problema como a la necesidad de diseñar una estructura coherente y funcional.

Finalmente, es relevante señalar que existen múltiples enfoques para abordar la problemática planteada. La solución desarrollada en el presente informe se considera adecuada y consistente con los requisitos establecidos; Sin embargo, con un mayor tiempo de análisis y el apoyo de experiencia profesional, sería posible alcanzar implementaciones alternativas que optimicen la complejidad del código, manteniendo o incluso mejorando los resultados obtenidos.

3.2. Análisis de resultados y objetivos

3.2.1. Resolución del Objetivo General

Se logró diseñar e implementar un modelo basado en programación orientada a objetos en C++, cumpliendo con los requerimientos planteados. La solución incluyó la elaboración de un diagrama de clases, así como la descripción detallada del funcionamiento de cada clase y de las relaciones existentes entre ellas.

Durante el desarrollo, se aplican principios fundamentales de la programación orientada a objetos, tales como abstracción, encapsulamiento, herencia y polimorfismo. Asimismo, el uso de estructuras como vectores y punteros resultó clave para la correcta gestión de la información y la implementación del sistema, permitiendo desarrollar una solución funcional orientada a la gestión y generación de rutinas de entrenamiento para un centro deportivo.

3.2.2. Objetivo específico 1

Se logró implementar correctamente las funcionalidades requeridas por el sistema, incluyendo la creación, actualización, eliminación, consulta y búsqueda de ejercicios, así como la generación de rutinas de entrenamiento.

Estas funcionalidades fueron centralizadas y gestionadas a través de la clase **Orquestador**, la cual actúa como eje principal del sistema. Mediante sus distintos métodos, se controla el flujo del programa, la interacción con el usuario y la coordinación entre las estructuras de datos. La combinación de programación orientada a objetos con el uso de vectores, punteros y funciones auxiliares permitió estructurar una solución eficiente y modular, demostrando la aplicabilidad de estas herramientas en la resolución de problemas reales.

3.2.3. Objetivo específico 2

El uso de punteros resultó fundamental para la gestión dinámica de memoria, permitiendo manipular objetos de manera eficiente dentro de estructuras como vectores. En conjunto con estos, se logró implementar una estructura tipo “base de datos” mediante las clases de colección, facilitando el almacenamiento y acceso a la información ingresada por el usuario.

Asimismo, la correcta utilización de estas herramientas permitió asegurar la liberación adecuada de memoria, aspecto clave en lenguajes como C++, donde la gestión de recursos recae directamente en el programador.

3.2.4. Objetivo específico 3

Durante el desarrollo del sistema, se presentó una problemática relevante asociada al manejo de memoria dinámica. Específicamente, al eliminar un ejercicio desde la colección, se generaban punteros colgantes en aquellas rutinas que hacían referencia a dicho objeto, lo que provocaba comportamientos inesperados e incluso la caída del programa.

Para abordar esta situación, se realizó un proceso de análisis en conjunto con una herramienta de inteligencia artificial, en este caso Copilot, evaluando distintas alternativas de solución. A partir de esta interacción, se identificó la necesidad de abstraer el problema a un nivel más simple, lo que permitió comprender que la solución consistía en generar copias independientes de los objetos en lugar de reutilizar las mismas direcciones de memoria.

Como resultado, se implementó una estrategia basada en la copia de objetos, permitiendo que cada rutina almacene instancias independientes de los ejercicios. Esto evitó la dependencia directa de los punteros originales y eliminó el problema de los punteros colgantes.

Este proceso evidencia que el uso de herramientas de inteligencia artificial puede ser un apoyo significativo en la resolución de problemas específicos, siempre que exista un razonamiento previo por parte del desarrollador. En este sentido, la inteligencia artificial no reemplaza el proceso de análisis, sino que actúa como un complemento que facilita la exploración de soluciones y la comprensión de conceptos complejos.

4. Referencias Bibliográficas

4.1. Referencias textuales

1. José L. Fernández, s.f. *¿Qué es un vector?* Recuperado de: https://www.fisicalab.com/apartado/concepto-de-vector
2. Geeks for Geeks, 2025. *¿Cómo funciona internamente un vector en C++?* Recuperado de: https://www.geeksforgeeks.org/cpp/how-does-a-vector-work-in-c/